

CS_33600

Streams

Processes use streams for all of their I/O (input/output) operations. Streams are an abstraction created by the operating system. A **stream** represents a sequence of bytes. The bytes can represent any kind of data, for example, text, images, video, audio. Processes use streams to move data into and out of hardware I/O devices (like the keyboard), files, or even other processes.

Streams are one directional. A process can only read from or write to a specific stream. If a process can read from a stream we say it is an **input stream**. If a process can write to a stream, we say it is an **output stream**.

In introductory programming courses, streams are mostly associated with files. A program reads or writes a stream of data from a file on a storage device. But we will see that streams are much more versatile. We will show how programs can read or write streams of data from other programs. In other words, we will see that streams can be used to implement Inter-process Communication.

- [https://en.wikipedia.org/wiki/Stream_\(computing\)](https://en.wikipedia.org/wiki/Stream_(computing))

The operating system creates the file and stream abstractions and then makes them available to programming languages for writing programs. Here are references to a few operating systems textbooks that explain how operating systems create these abstractions.

- [I/O Devices](#) from [Operating Systems: Three Easy Pieces](#)
- [Files and Directories](#) from [Operating Systems: Three Easy Pieces](#)
- [Chapter 4, Files and file systems](#) from [Think OS](#)
- [POSIX File API](#) from [Operating Systems and Middleware](#)

In this document we will emphasize how streams are used to build up complex command-lines. In another document we will look at how we can use the Java programming language to write code that uses streams.

All the example programs mentioned in this document are in the sub folder called "filter_programs" from the following zip file.

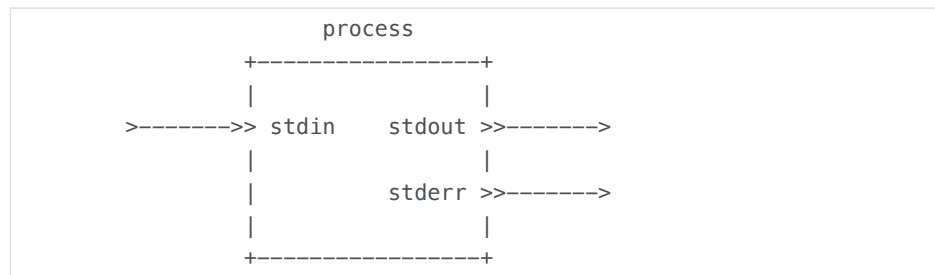
- http://cs.pnw.edu/~rlkraft/cs33600/for-class/streams_and_processes.zip

Standard I/O streams

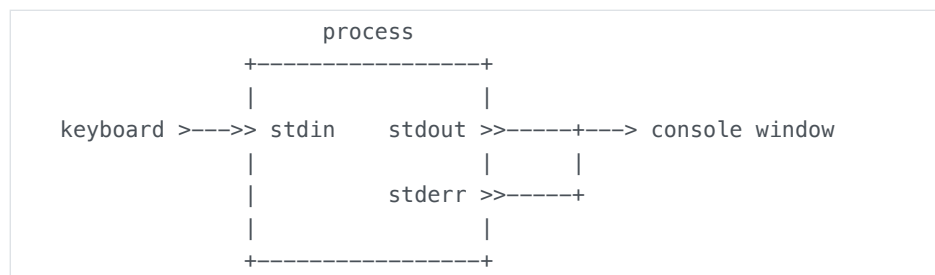
When a process is created by the operating system, the process is always supplied with three open streams. These three streams are called the **standard streams**. They are

- standard input (stdin)
- standard output (stdout)
- standard error (stderr)

We can visualize a process as an object with three "connections" where data (bytes) can either flow into the process or flow out from the process.



A console application will usually have its `stdin` stream connected to the computer's keyboard and its `stdout` and `stderr` streams connected to the console window.



It is important to realize that the above picture is independent of the programming language used to write the program which is running in the process. Every process looks like this. It is up to each programming language to allow programs, written in that language, to make use of this setup provided by the operating system.

- https://en.wikipedia.org/wiki/Standard_streams

Every **operating system** has its own way of giving each process access to the internal data structures the operating system uses to keep track of what each standard stream is "connected" to.

The Linux operating system gives every process three **file descriptors**,

```
#define STDIN_FILENO 0, STDOUT_FILENO 1, STDERR_FILENO 2
```

Linux provides the `read()` and `write()` system calls to let a process read from and write to these file descriptors.

The Windows operating system gives every process three **handles**. We retrieve the handles using the `GetStdHandle()` function with one of these input parameters.

```
STD_INPUT_HANDLE, STD_OUTPUT_HANDLE, STD_ERROR_HANDLE
```

Windows provides the `ReadFile()` and `WriteFile()` system calls to let a process read from and write to these handles.

Every **programming language** must have a way of representing the three standard streams and every language must provide a way to read from an input stream and a way to write to an output stream.

For example, here is how the three standard I/O streams are represented by some common programming languages.

```
Java uses Stream objects.
java.io.InputStream  System.in
java.io.OutputStream System.out
java.io.PrintStream  System.err
These are static fields in the java.lang.System class.
```

```
Standard C uses pointers to FILE objects.
FILE* stdin;
FILE* stdout;
FILE* stderr;
These are defined in the stdio.h header file.
```

```
Python uses text File objects.
sys.stdin
sys.stdout
sys.stderr
These are in the sys module.
```

```
C++ uses stream objects.
istream std::cin;
ostream std::cout;
ostream std::cerr;
These are defined in the <iostream> header.
```

```
.Net uses Stream objects.
System.IO.TextReader Console.In
System.IO.TextWriter Console.Out
System.IO.TextWriter Console.Error
These are static fields in the System.Console class.
```

Most programming languages define their basic I/O functions to automatically work with the standard input and output streams. For example, in almost every programming language, the basic `print` function writes to the standard output stream (and a slightly different print function is used to write to the standard error stream).

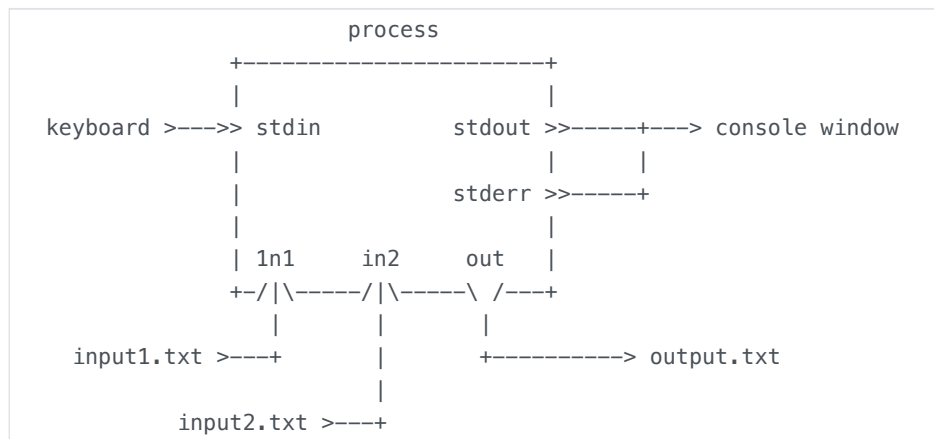
The C language provides functions like `getchar()`, `scanf()`, and `fscanf()` to read from `stdin` and it provides `printf()` and `fprintf()` to write to `stdout` and `stderr`. On a Windows computer, the C language's `printf()` function will be implemented using Window's `WriteFile()` system call with the `STD_OUTPUT_HANDLE` handle. On a Linux computer, the C language's `printf()` function will be implemented using Linux's `write()` system call with the `STDOUT_FILENO` file descriptor.

Here are a few references to how the Java, C, Python, C++, and C# languages provide access to the three standard streams.

Each one of these references is part of a larger documentation system that it's a good idea to be familiar with, for example "Java Javadocs", "Python docs", "Linux man pages", "Microsoft Learn", etc.

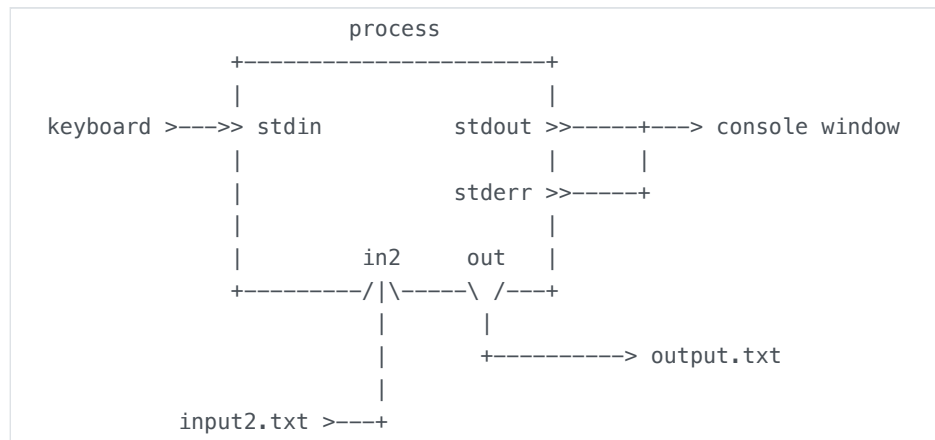
- <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/System.html#summary>
- <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/io/FileDescriptor.html>
- <https://man7.org/linux/man-pages/man3/stdio.3.html>
- https://sourceware.org/glibc/manual/latest/html_node/Standard-Streams.html
- <https://en.cppreference.com/w/c/header/stdio.html>
- <https://cplusplus.com/reference/cstdio/>
- <https://docs.python.org/3/library/sys.html#sys.stdin>
- <https://en.cppreference.com/w/cpp/header/iostream.html>
- <https://cplusplus.com/reference/iostream/>
- <https://learn.microsoft.com/en-us/dotnet/api/system.console>

Every operating system provides a way for processes to **open** new streams. For example, in the following picture, a process, while it was running, opened three new streams, two input streams and one output stream. All three streams are connected to files.



This process can now read data from any of its three input streams and it can write data to any of its three streams. For example, it might copy data from the two input files into the output file.

After the process has read all the data it needs from the file "input1.txt", the process can **close** the stream.



As long as a process is running, it can continue to open and close input and output streams. Opening and closing streams to files is what most introductory programming textbooks cover in their chapters on file I/O.

So far we have been using the words "stream" and "file" almost interchangeably. There are several reasons for this blurring of the two ideas. First, there is the Unix/Linux idea that "everything is a file". Second, there is the quirk in the C Language that "The connection to an open file is represented either as a stream or as a file descriptor." Third, in the C language, the data type that represents a stream is called `FILE`. Fourth, in the Linux operating system, the stream abstraction is built on Linux's lower level "file descriptor" interface.

Here are some links that (try to) explain this "stream" vs. "file" distinction.

- https://en.wikipedia.org/wiki/Everything_is_a_file
- <https://opensource.com/life/15/9/everything-is-a-file>
- <https://unix.stackexchange.com/questions/141016/141020#141020>
- https://sourceware.org/glibc/manual/latest/html_node/I_002fO-Concepts.html
- https://sourceware.org/glibc/manual/latest/html_node/Streams.html
- https://sourceware.org/glibc/manual/latest/html_node/Streams-and-File-Descriptors.html

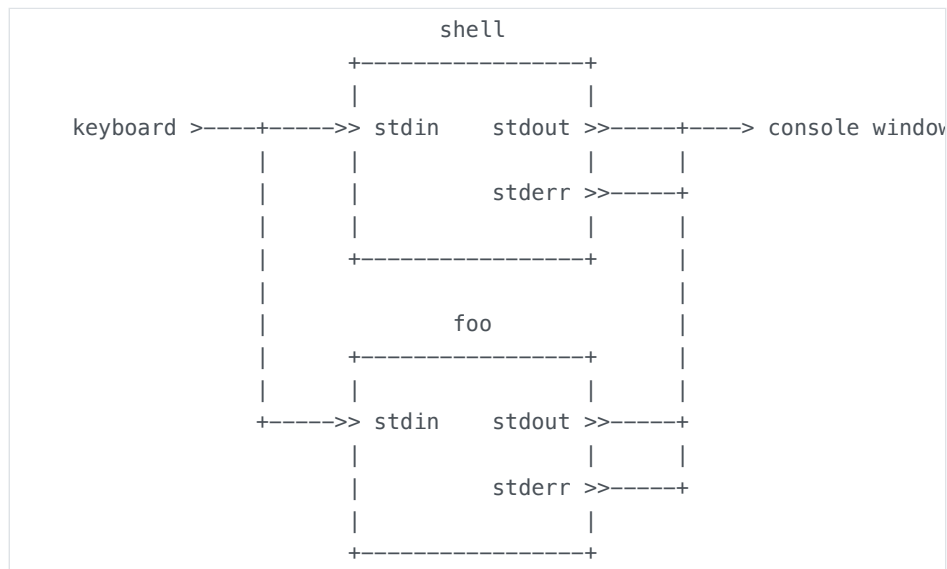
I/O redirection

Every process is created by the operating system at the request of some other process, the parent process. When the parent process asks the operating system to create a child process, the parent must tell the operating system how to "connect" the child's three standard streams. The parent telling the operating system how to connect the child's three standard streams is usually referred to as **I/O redirection**.

At a shell command prompt, if we type a command like this,

```
> foo
```

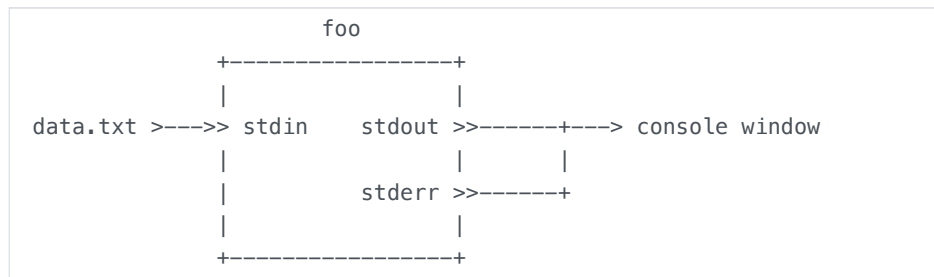
the shell process (cmd on Windows, or bash on Linux) is the parent process. This command tells the shell process to ask the operating system to create a child process from the `foo` program and have the `foo` process inherit (share) all three of its standard streams with the shell process.



At a shell command prompt, if we type a command like this,

```
> foo < data.txt
```

the shell process will ask the operating system to create a child process from the `foo` program. In addition, the shell process will instruct the operating system to redirect the child process's standard input stream to the file "data.txt". The stdout and stderr streams will have their default (shared) connections. When `foo` runs, its stream connections will look like this.



At a shell command prompt, if we type a command like this,

```
> foo > result.txt
```

the shell process will ask the operating system to create a child process running the `foo` program. In addition, the shell process will instruct the operating system to redirect the child process's standard output to the file "result.txt". The stdin and stderr streams will have their default (shared) connections. When `foo` runs, its stream connections will look like this.



If we type a command like this,

```
> foo > result.txt < data.txt
```

the shell process will ask the operating system to create a child process running the `foo` program but this time the shell will ask the operating system to redirect the standard output stream to the file "result.txt" *and* redirect the standard input stream to the file "data.txt". The stderr stream will have its default (shared) connection. When the `foo` process runs, its connections look like this.



An important point is that the `foo` process does *not* know that its standard streams have been redirected. It cannot tell if its standard output stream is connected to the console (the default) or to a file. If its standard output is redirected to a file, then `foo` does file I/O without realizing it.

Many years ago, computers were connected to teletype machines, and the `print` function literally printed messages on paper. Today, we think of `print` as displaying text in a console window. But if the standard output stream is redirected to a file, then `print` is "printing" to a file and if the output stream is redirected to another process, then `print` is "printing" to a process. In both cases `print` is not really "displaying" text so that we can see it.

The `print` function isn't really meant to "print". It is a way to transmit textual data over a stream from a process to a destination. Later in the course we will look at examples where we `print` text over a network to another computer.

The order in which we place redirections in the command-line does not matter. The following two commands are equivalent.

```
> foo > result.txt < data.txt  
> foo < data.txt > result.txt
```

When we use the input redirection operator, `<`, if the specified input file does not exist, then we get an error message and the command-line fails.

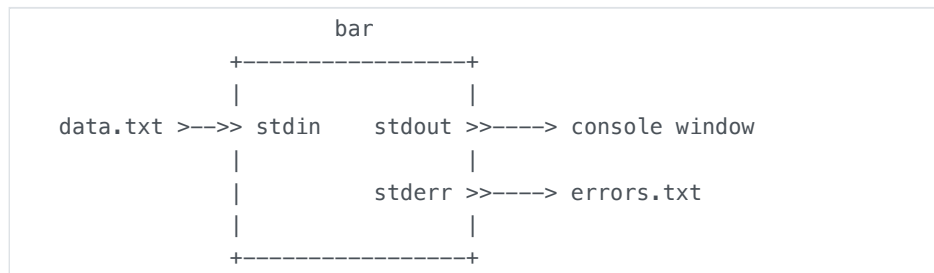
When we use the output redirection operator, `>`, if the specified output file does not exist, then the operating system creates an empty file for us with that name. However, be careful. If the specified output file does exist, then it is emptied of all its contents, and the command-line is given the empty file, so we lose any data that was in the specified output file.

There is a useful alternative to the `>` output redirection operator. The `>>` append output redirection operator will, like `>`, create the specified output file if it does not exist, but instead of emptying an existing file, this operator writes new data at the end of the previous data in the file. One important use of this operator is for one file to accumulate results from several command-lines.

There is an operator, `2>`, that tells the shell process to redirect the standard error stream of a process. The following command-line,

```
> bar < data.txt 2> errors.txt
```

tells the shell process to ask the operating system to create a child process from the `bar` program, redirect the child's standard input stream to the file "data.txt", and redirect the child's standard error stream to the file "errors.txt". The child's standard output stream will be connected to the console window. When the `bar` process runs, its connections look like this.



The order of the redirections in the command-line does not matter. The following two commands are equivalent.

```
> bar < data.txt 2> errors.txt
> bar 2> errors.txt < data.txt
```

In fact, the following command-lines are all equivalent.

```
> bar < data.txt > output.txt 2> errors.txt
> bar > output.txt 2> errors.txt < data.txt
> bar > output.txt < data.txt 2> errors.txt
```

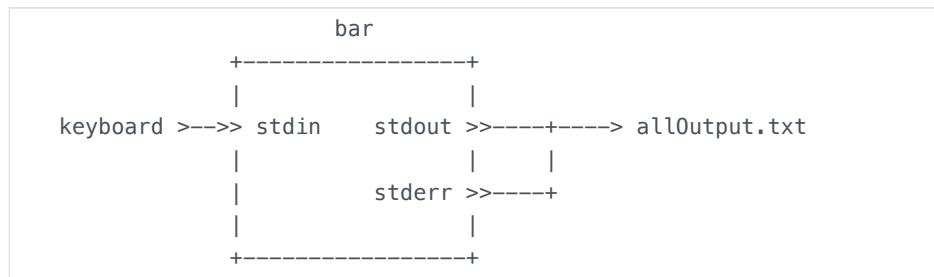
What if we want to redirect both the standard output and standard error streams to a single file? The following command-line does not work.

```
> bar > allOutput.txt 2> allOutput.txt
```

The Linux bash shell allows us to use the `&>` redirection operator.

```
$ bar &> allOutput.txt
```

This creates the following picture.

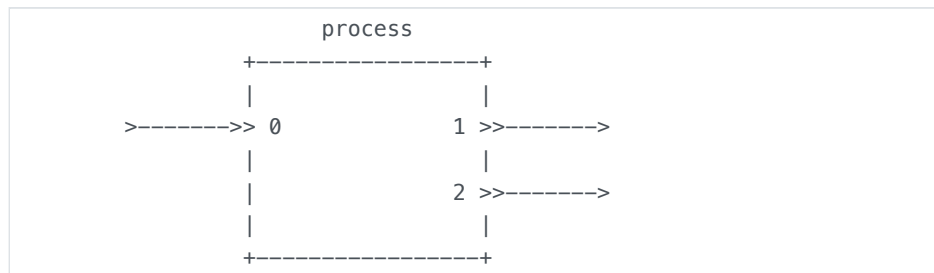


With the Windows cmd shell, we need to use this slightly more complex command (which also works with bash).

```
> bar > allOutput.txt 2>&1
```

This command tells the operating system to redirect the standard output stream to the file "allOutput.txt" and then, in addition, redirect the standard error stream to the same place as the standard output stream. The `2>&1` operator must be at the end of the command-line.

Where do the numbers 1 and 2 in the I/O redirection operators come from? They are from the Unix operating system's implementation of file I/O. In Unix (and in Linux) every open file is given a positive integer number called a **file descriptor**. The file descriptor numbers are used by all the Unix (and Linux) file I/O functions. When a process is created, its standard input, output, and error streams are given the file descriptors 0, 1, and 2, respectively.

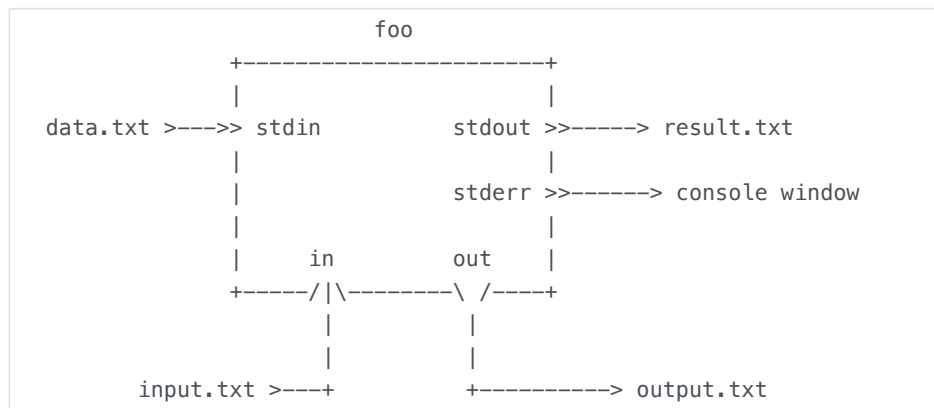


The bash and cmd shells use these file descriptor numbers as part of their I/O redirection operators. This is an example of a "leaky abstraction". The shell program is supposed to let us manipulate processes and files with out knowing about the details of how the underlying operating system handles processes and files. The Windows operating system does not even use file descriptors, but it still exposes them in the syntax of the `cmd` shell (in order to be

consistent with bash). A leaky abstraction is when a lower level implementation detail appears in the interface of a higher level abstraction.

- https://en.wikipedia.org/wiki/File_descriptor
- <https://man7.org/linux/man-pages/man2/open.2.html>
- https://en.wikipedia.org/wiki/Leaky_abstraction
- <https://www.joelonsoftware.com/2002/11/11/the-law-of-leaky-abstractions/>

Do not confuse I/O redirection with the idea of opening a new stream to a file. The above `foo` process, that has its stdin redirected to the file "data.txt", and its stdout redirected to the file "result.txt", can still open new streams connected to other files.



Opening (and closing) new file streams does not change the fact that this process has had its standard input and output streams redirected.

Here are several links to general explanations of I/O redirection in both bash and cmd.

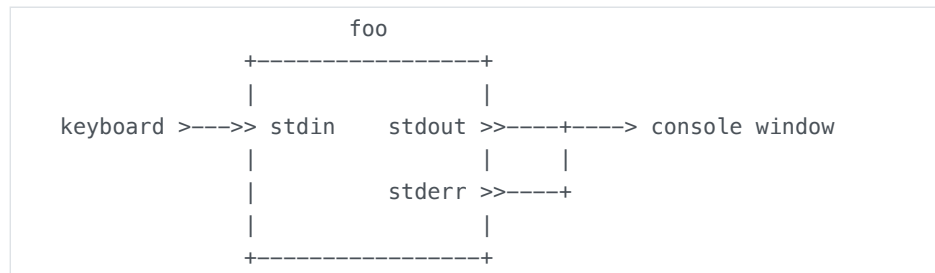
- [https://en.wikipedia.org/wiki/Redirection_\(computing\)](https://en.wikipedia.org/wiki/Redirection_(computing))
- <https://www.linfo.org/redirection.html>
- https://linuxcommand.org/lc3_lts0070.php
- https://www.gnu.org/software/bash/manual/html_node/Redirections.html
- <https://man7.org/linux/man-pages/man1/bash.1.html#REDIRECTION>
- <https://ss64.com/nt/syntax-redirection.html>
- [https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-xp/bb490982\(v=technet.10\)](https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-xp/bb490982(v=technet.10))

Shared streams

At a shell command prompt, if we type this command-line,

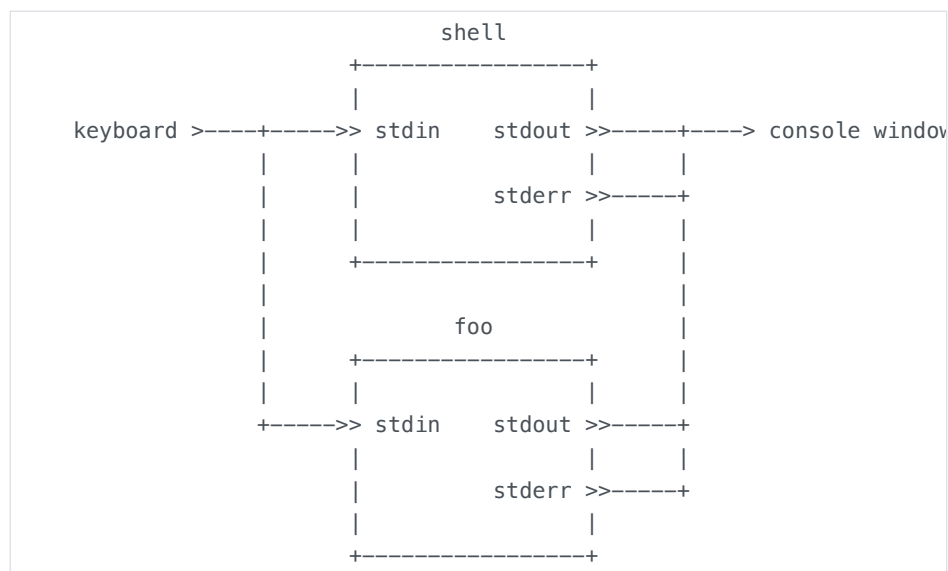
```
> foo
```

then we are asking the shell process to create and run a `foo` process. The shell process (cmd on Windows, or bash on Linux) is the parent process and `foo` is its child process. The shell process causes the `foo` process to have its standard streams connected in the following, usual, way.



But this picture is incomplete. It does not show the relationship between the `foo` process and the shell process, its parent process. The shell process is itself a command-line program, so it uses the keyboard for its input and the console window for its output.

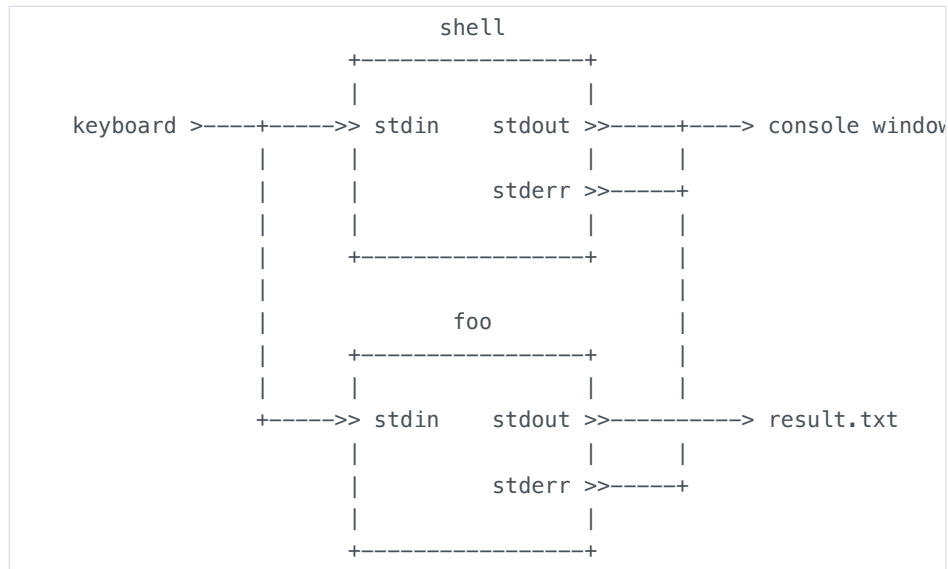
Here is how the two processes are related to each other. The two process "share" the input stream for the keyboard and they share the output stream to the console window.



If, at a shell command prompt, we type this command-line,

```
> foo > result.txt
```

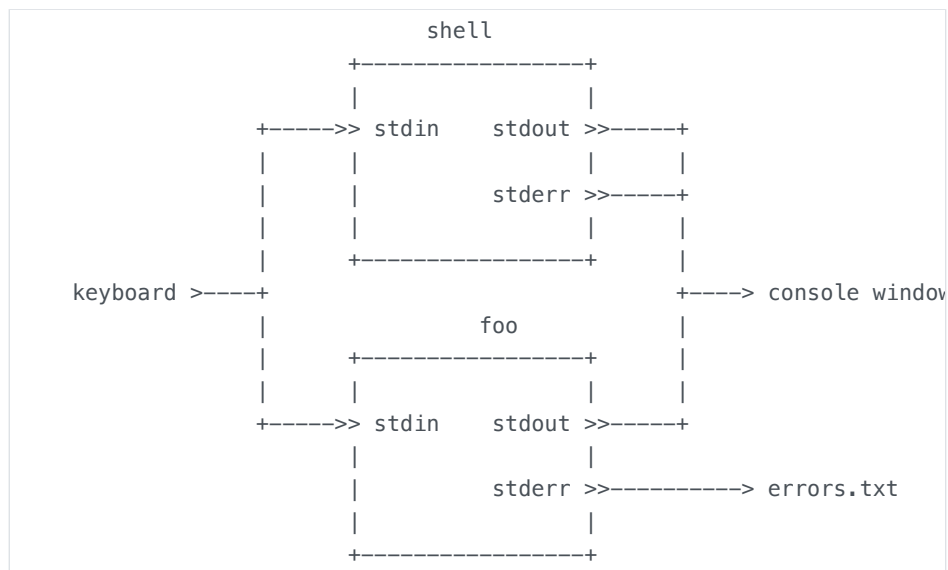
then the shell process is the parent process and the `foo` process is the child process. The child has its standard output stream redirected to a file, but it uses the default input stream (and default error stream), which it shares with the shell process. The two processes and their streams will look like this.



If, at a shell command prompt, we type this command-line,

```
> foo 2> errors.txt
```

then we get the following picture. The `foo` process shares its standard input and output streams with the shell process.

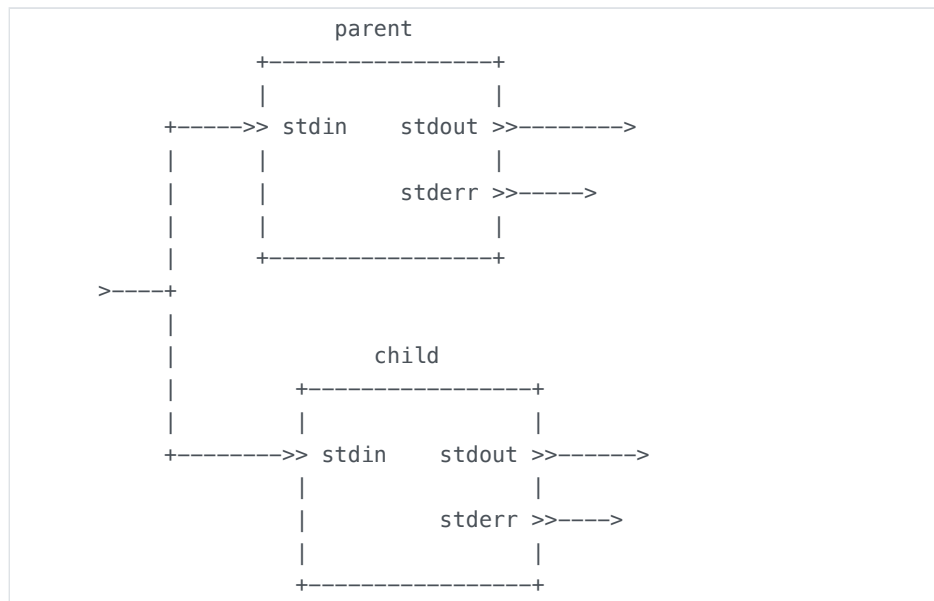


When two processes share a stream, it is usually the case that one of the two processes is idle while the other process uses the shared stream (the idle process will often be waiting for the other process to

terminate). If two processes are simultaneously using a shared stream, the results can be confusing and unpredictable.

If two processes simultaneously use an output stream, then their outputs will be, more or less, randomly intermingled in the stream's final destination. This can lead to unusable results.

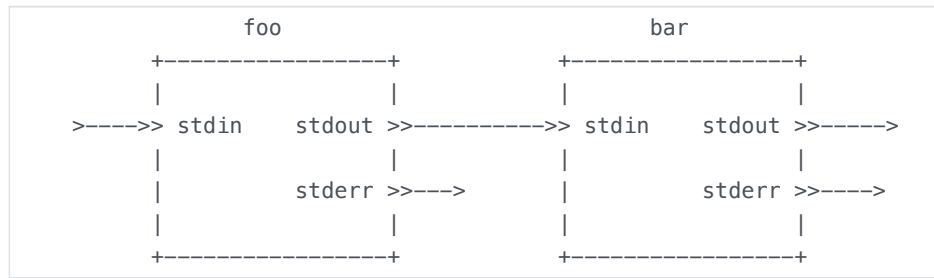
If two processes simultaneously use an input stream, as in the following picture, then it is **not** the case that every input byte flows into each process. Each input byte can only be consumed by *one* of the two processes. Which process gets a particular byte of input depends on the ordering of when each process calls its `read( )` function on the input stream. This is almost never a desirable situation. Processes almost never simultaneously use a shared input stream. Shared input streams are common, but the two processes almost always have a way to synchronize their use of the stream so that they are never reading from it simultaneously. The most common way for two processes to share an input stream is for the parent process to wait for the child process to terminate. Then the parent process can resume reading from the input stream.



Pipes

So far, we have seen that streams can connect a process to either a file or an I/O device (like the keyboard or a console window).

It would be useful if the output stream of one process could be connected to the input stream of another process, something like this.



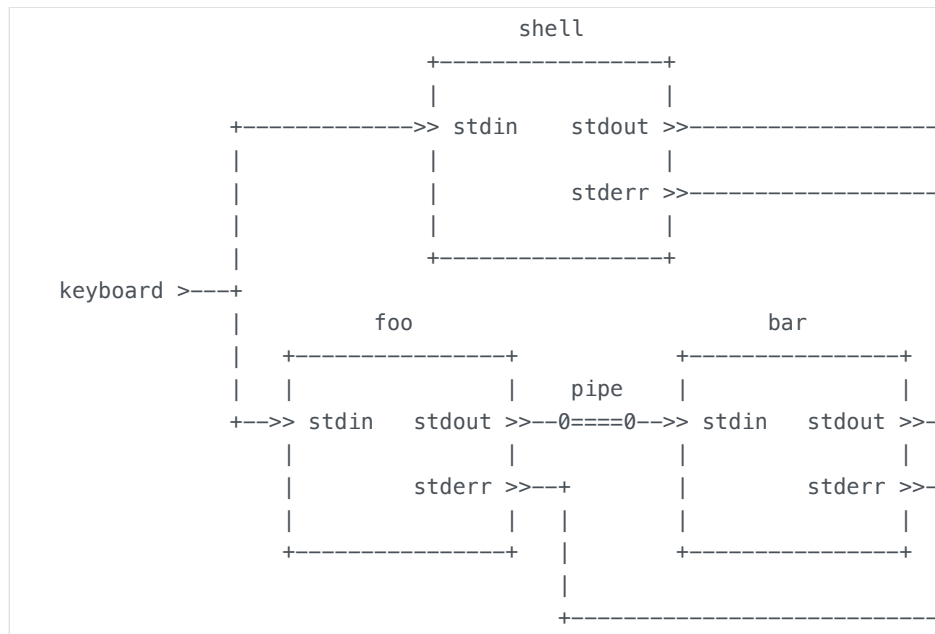
This picture is supposed to represent the idea that the `foo` process can send information to the `bar` process by `foo` printing to its standard output stream and `bar` reading from its standard input stream.

The above picture is not possible. The operating system does not allow the output stream of one process to be connected directly to the input stream of another process. But the idea is useful, so the operating system provides an object, called a **pipe**, that can be placed between two processes, and can allow the output from one process to be used as input to another process.

Consider the following command-line.

```
> foo | bar
```

The `|` character is (in the context of a command-line) called the **pipe symbol**. This command-line asks the shell process to create two child processes, one from the `foo` program and the other from the `bar` program. In addition, the shell process will ask the operating system to create a **pipe** object and have the standard output stream of the `foo` process redirected to the input of the pipe, and have the standard input stream of the `bar` process redirected to the output of the pipe. This create a picture that looks like the following. Notice that `foo` shares the keyboard with the shell, and `bar` shares the console window with the shell. Also notice that the error stream from `foo` is combined with the output and errors streams from both the shell and `bar`.

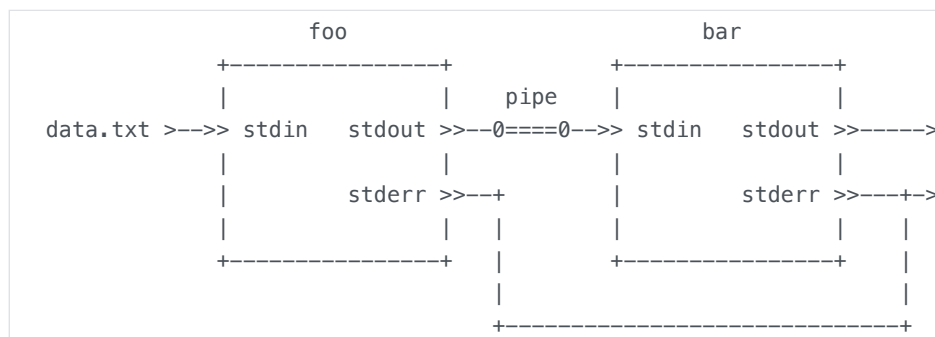


The shell process will wait for *both* child processes to terminate before the shell will resume using the shared keyboard and console window.

If we type a command-line like this,

```
> foo < data.txt | bar > result.txt
```

then the shell process will ask the operating system to create two child processes, one from the `foo` program and the other from the `bar` program. In addition, the shell process will ask the operating system to create a **pipe** object and have `stdout` of the `foo` process redirected to the input of the pipe, and have `stdin` of the `bar` process redirected to the output of the pipe. Finally, the shell process will ask the operating system to redirect the `foo` process's standard input to the file `data.txt` and redirect the `bar` process's standard output to the file `result.txt`. While this command is executing, it looks like the following picture (this picture doesn't show the parent shell process and its streams).



In the above command, the two processes, `foo` and `bar`, are running simultaneously (in parallel) with each other. The pipe object acts as a "buffer" between the two processes. Whenever the `foo` process writes something to its output stream, that something gets put in the pipe buffer. When the `bar` process wants to read from its input stream, it does so by removing data from the pipe buffer.

If the `foo` process writes data into the pipe buffer faster than the `bar` process can read data out of the pipe buffer, then data accumulates in the buffer. If the `foo` process writes data so fast that it fills up the buffer, then the operating system makes the `foo` process "block" and wait for the `bar` process to read some data from the pipe buffer. When the `bar` process reads some data from the buffer, freeing up space in the buffer, then the operating system "unblocks" the `bar` process so that it can resume writing data into the buffer.

If the `bar` process reads data out of the pipe buffer faster than the `foo` process can write data into the buffer, then the `bar` process will often find the pipe empty when `bar` wants to read some data. In that case, the operating system "blocks" the `bar` process and makes it wait until some data shows up in the pipe. When the `foo` process writes some data to the pipe, then the operating system "unblocks" the `bar` process so that it can resume reading data from the pipe buffer. (You should compare this to what happens when a process tries to `pop()` and empty stack data structure.)

When the `foo` process detects the end-of-file (eof) condition on its input stream, and it knows that it has no more data to process, the `foo` process should close its output stream. The operating system will then cause the end-of-file condition to become true for the input stream of `bar`. The `bar` process then knows that it has reached the end of its data. After closing its output stream, the `foo` process will probably terminate, but doesn't have to.

When `foo` closes its output stream, it may be that some data remains in the pipe buffer. In that case `bar` can continue to read until it has emptied the pipe. When `bar` reads the last byte of data from the pipe buffer, then the operating system will tell `bar` that it has reached the end-of-file on its input stream.

If `foo` is no longer sending data to `bar` but `foo` never closes its output stream, then `bar` (and the operating system) will not know that there is no more data coming through the pipe and `bar` will block (forever) waiting for `foo` to write something into the pipe buffer. Since `bar` will be blocked but still running, the parent process will also stay blocked (forever) waiting for `bar` to terminate. So the parent and `bar` will wait forever. A situation like this is called a "deadlock". It is essential that the `foo` process close its output stream when it no longer has data to send into the pipe. If the `foo` process terminates, the operating system may close the output stream for `foo`, but it's not wise to depend on the operating system. The `foo` process should close its own output stream.

It is possible for the `bar` process to terminate before the `foo` process does. In that case, it is not a good idea to let the `foo` process fill up the pipe buffer and then block (forever). If the `bar` process terminates (or, if the `bar` process closes its input stream) and the `foo` process then writes data into the pipe buffer, the operating system will send an I/O exception to the `foo` process. Any data left in the pipe buffer is considered lost.

This coordination that we just described, between the two processes on the ends of a pipe, is referred to in computer science as "bounded buffer synchronization" or the "producer-consumer problem".

Here are a few links to explanations of bounded-buffer synchronization.

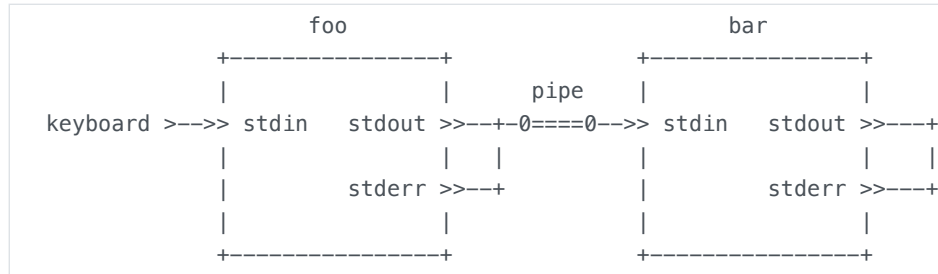
- https://en.wikipedia.org/wiki/Bounded-buffer_problem
- <https://www.baeldung.com/cs/bounded-buffer-problem>
- <https://www.baeldung.com/cs/buffer>
- <https://pages.cs.wisc.edu/~remzi/OSTEP/threads-cv.pdf#page=6>

In the Linux bash shell there is another version of the pipe operator, the `|&` operator. If we type this command-line,

```
$ foo |& bar
```

then the bash process will ask the operating system to create `foo` and `bar` child processes, then bash will ask the operating system to create a `pipe` object and have `stdout` and `stderr` of the `foo`

process redirected to the input of the pipe, and have `stdin` of the `bar` process redirected to the output of the pipe. While this command is executing, it looks like the following picture (this picture doesn't show the bash shell process and its streams).



This would be useful if the `bar` process needs to know about and handle errors from the `foo` process. The Windows `cmd` shell does not have this version of the pipe operator but it can be implemented with this slightly more complex command-line (which works on Linux too).

```
> foo 2>&1 | bar
```

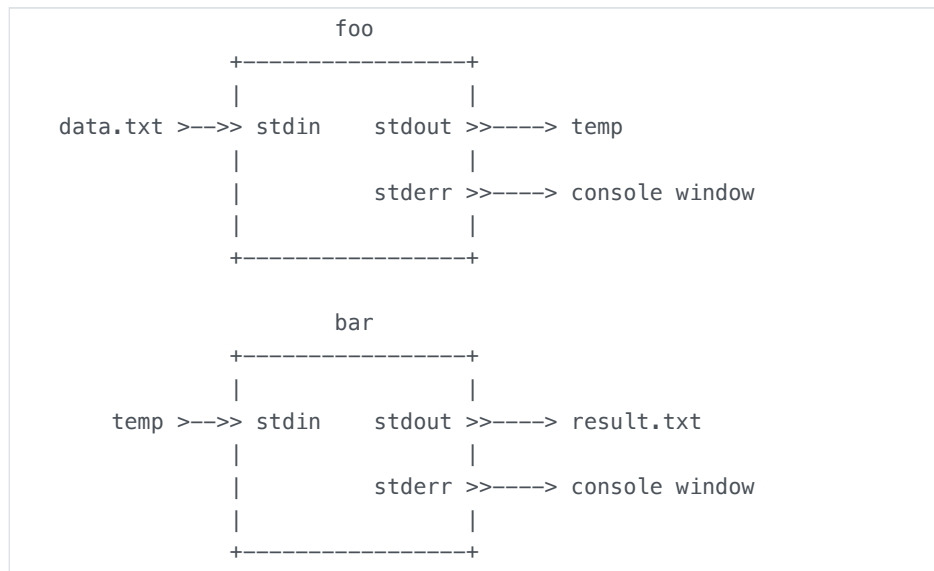
Here is another way to think about the shell's pipeline operator. The shell process could run the two programs, `foo` and `bar`, sequentially, one after the other. In other words, the shell process could interpret this command,

```
> foo < data.txt | bar > result.txt
```

as the following three commands.

```
> foo < data.txt > temp
> bar < temp > result.txt
> del temp
```

These three commands would have a picture that looks like this.



First the `foo` process runs with its output stored in a temporary file called "temp". Then the `bar` process runs with its input coming from the "temp" file. Then the "temp" file is deleted.

Notice that this sequential interpretation of the pipeline command might be considerably slower than the parallel interpretation. And since the sequential interpretation needs to store all the intermediate data in a temporary file, the sequential interpretation may require far more storage space than the parallel interpretation.

One final remark. Do not confuse the shell's pipe operator, the `|` character, with the operating system's `pipe` object. The operating system's `pipe` object is an object provided by the OS to efficiently implement one kind of inter-process communication. The shell's pipe operator is a way for the shell's user to request that two processes communicate. The shell may or may not implement its pipe operator using an OS `pipe` object (see the last few paragraphs).

Here is the documentation for the Linux and Windows operating system functions that create `pipe` objects.

- <https://man7.org/linux/man-pages/man2/pipe.2.html>
- <https://learn.microsoft.com/en-us/windows/win32/api/namedpipeapi/nf-namedpipeapi-createpipe>

Here are references to the bash and cmd pipe operators.

- https://en.wikipedia.org/wiki/Anonymous_pipe
- [https://en.wikipedia.org/wiki/Pipeline_\(Unix\)](https://en.wikipedia.org/wiki/Pipeline_(Unix))
- https://en.wikipedia.org/wiki/Vertical_bar#Pipe
- https://www.gnu.org/software/bash/manual/html_node/Pipelines.html
- https://linuxcommand.org/lc3_lts0070.php#:~:text=Pipelines
- <https://ss64.com/nt/syntax-redirectation.html#:~:text=Pipes%20and%20CMD.exe>
- [https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-xp/bb490982\(v=technet.10\)](https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-xp/bb490982(v=technet.10))
- <https://en.wikipedia.org/wiki/Foobar>

Filters and pipelines

The example programs mentioned in this section are all in the sub folder called "filter_programs" from the following zip file.

- http://cs.pnw.edu/~rlkraft/cs33600/for-class/streams_and_processes.zip

Pipes are useful. Their usefulness comes from combining them with a kind of program called a filter. When pipes and filters are combined together, we call those systems **data pipelines**.

- [https://en.wikipedia.org/wiki/Pipeline_\(software\)](https://en.wikipedia.org/wiki/Pipeline_(software))
- <https://aws.amazon.com/what-is/data-pipeline/>
- <https://learn.microsoft.com/en-us/azure/architecture/patterns/pipes-and-filters>
- <https://www.ibm.com/think/topics/data-pipeline>
- <https://dataengineering.wiki/Concepts/Data+Pipeline>

A **filter** is a program that reads data from its standard input stream, does some kind of operation on the data, and then writes the converted data to its standard output stream.

- [https://en.wikipedia.org/wiki/Filter_\(software\)](https://en.wikipedia.org/wiki/Filter_(software))

Shell based filter programs are related to "filter functions" in functional programming languages.

- [https://en.wikipedia.org/wiki/Filter_\(higher-order_function\)](https://en.wikipedia.org/wiki/Filter_(higher-order_function))
- <https://learn.microsoft.com/en-us/dotnet/fsharp/language-reference/functions/#pipelines>

- <https://alvinalexander.com/downloads/fpsimplified-free-preview.pdf#page=181>

Data pipelines are usually implemented on a very large scale, processing gigabytes of data. But pipelines can also be useful on a small scale, while working with files on your personal computer. The Windows and Linux operating systems both come with filter programs installed. Filter programs can be used, for example, to sort, search, format, or convert files.

- https://linuxcommand.org/lc3_lts0070.php#:~:text=Filters
- [https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-xp/bb490905\(v=technet.10\)](https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-xp/bb490905(v=technet.10))

In an enterprise environment, pipelines and filters are used for processing huge amounts of data. In a desktop environment, pipelines and filters are used to quickly do small tasks like formatting or converting files. In this section we will use pipelines and filters as a way to understand command-line operators and to reinforce your understanding of processes and streams. We will also use pipelines and filters to demonstrate some subtle concepts about operating systems and I/O.

To get a feel for working with pipes and filters, it helps to experiment with actual filter programs. In this section we will work with a collection of simple filter programs, written in Java and C, contained in the folder "filter_programs".

In the "filter_programs" folder there are Java programs that act as filters. They are all short programs that do simple manipulations of their input text. Look at the source code to these programs. Compile them and then run them using command-lines like the following.

```
> java Reverse < Readme.txt > result.txt
> java Double < Readme.txt | java Reverse
> java Double | java ToUpperCase | java Reverse
> java ShiftN 2 | java ToUpperCase | java Reverse
> java Twiddle < Readme.txt | java ToUpperCase | java Double | java
> java Find pipe < Readme.txt | java CountLines
> java OneWordPerLine < Readme.txt | Find pipe | java CountLines
```

Let's look a few specific programs to see how they manipulate input data to produce output data. Run the `ToUpperCase` program.


```
> java ToUpperCase
```

When you run this program, it will wait for you to type, using the keyboard, some text into its standard input stream. Notice that you need to tap the **Enter** key to send your input from the keyboard to the process. The process will print the result from your first line of input text in the console window, and then wait for you to type another line of input text. This program does not "prompt" you for input, it just waits for you to type.

When you don't want to type any more input, use Ctrl-z in Windows, or Ctrl-d in Linux, to close the program's input stream. Do **not** use Ctrl-c, because that kills the program. Killing the program prevents it from finishing up whatever work it needs to do after it has read the last of its input data. Most simple programs will work correctly if you use Ctrl-c, but not all of them. For example, try typing these three lines into your console window.

```
> java LongestLineReversed
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx012345
^c
```

Then try these three lines.

```
> java LongestLineReversed
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx012345
^z
```

When you terminate **LongestLineReversed** with Ctrl-c, the process does not have enough time to finish its last for-loop (look at the source code file, "LongestLineReversed.java"). The Ctrl-c "kill TERM signal" does not take affect instantly. The program being terminated gets to run for a small amount of time before the kill signal takes affect (that's why simple programs seem to work OK when you use Ctrl-c).

- <https://man7.org/linux/man-pages/man1/kill.1.html>

Let's go back to the "ToUpperCase" example and ask a simple question.

```
> java ToUpperCase
```

Why doesn't the program help the user with a simple "prompt" for input? It doesn't do that because that would be a big mistake. A "prompt" would ruin the program's utility as a filter program!

If a program wants to "prompt" a user, the program must write the prompt string to its standard output stream. But then that prompt becomes part of the program's output and it gets mixed up with the program's primary output which is the transformed version of its input data.

Consider a pipeline like this.

```
> java LineNumbers < Readme.txt | java Find pipe | java ToUpperCase
```

If the `ToUpperCase` program added a prompt to its output, that would show up in the output listing of the results from the `Find` program, which would not make sense.

A filter program must read from its standard input stream and only write the transformed version of its input data to the output stream. If the filter program needs to report any error messages, that output should go to the filter program's standard error stream.

Another way to think about this is that the vast majority of programs do not have a "user". The vast majority of programs get their data not from a "user", but from some other program that feeds it data through its standard input stream. Similarly, most programs do not output messages to a "user", they output data to another program connected to their standard output stream. And error messages do not get seen by a "user", they get sent to a log file. The program that is receiving data does not need to, and shouldn't, "prompt" the program that is sending data. The program that is receiving data should just wait for new data to arrive. The program that is sending data will send new data when it is ready.

Lets try another one of the filter programs.

```
> java CountLines
```

Notice that there is no output until the input is terminated (end-of-file) using either Ctrl-z on Windows or Ctrl-d on Linux. This program needs to see every line of input before it can report how

many lines it counted. The program does not produce any output until after there is no more input (look at the source code file, "CountLines.java").

Lets try another filter program.

```
> java MakeOneLine
```

If you run this program interactively, it doesn't seem to do anything useful. But run it with an input file redirected into its standard input stream.

```
> java MakeOneLine < Readme.txt  
> java MakeOneLine < Readme.txt | java CountLines
```

Some of the programs in the "filter_programs" folder are not filters. They are programs that are useful for testing pipelines and demonstrating pipeline concepts.

For example, the "Source.java" program does not read any data from its input stream and writes just one line of text to its output stream.

```
> java Source
```

This program is useful as the first program in a pipeline, as a source of data.

```
> java Source | java EchoN 15 | java LineNumbers
```

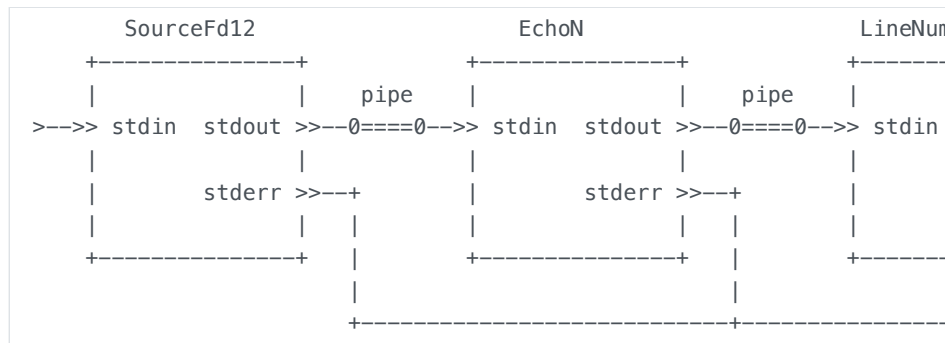
The program "SourceFd12.java" is similar to "Source.java" but it writes one line of text to each of its standard output and standard error streams (the streams with file descriptors 1 and 2, hence the name of the program).

```
> java SourceFd12
```

This can be used to demonstrate how a pipeline handles the error stream.

```
> java SourceFd12 | java EchoN 15 | java LineNumbers
```

Notice how the error stream does not go through the pipeline programs. Remember how the streams of a pipeline are connected. The standard error stream from `SourceFd12` is connected directly to the (shared) console window.



What output do you think this command-line will produce?

```
> java SourceFd12 | java SourceFd12 | java LineNumbers
```

We can redirect stdout and stderr to different files.

```
> java SourceFd12 > out.txt 2> err.txt
```

Look in the files "out.txt" and "err.txt".

What does the following command-line do? Draw a picture of the stream connections for the command-line.

```
> java SourceFd12 2> err1.txt | java EchoNFd12 5 2> err2.txt | java
```

We can redirect stdout and stderr to the same file. Another way to put this is that we can merge the standard output and error streams. Notice how stderr is redirected (merged) to stdout *after* stdout is redirected.

```
> java SourceFd12 > out-err.txt 2>&1
```

Look in the file "out-err.txt". Compare the last command-line with this one. Draw a picture of all the stream connections for each of these two command-lines.

```
> java SourceFd12 > out-only.txt
```

Exercise: Try to explain why this command-line doesn't do what you think it might. (Hint: order matters.)

```
> java SourceFd12 2>&1 > out-err.txt
```

Exercise: Compare the following three command-lines. Draw a picture of all the stream connections for each one.

```

> java SourceFd12      | java EchoNFd12 5 > temp1.txt
> java SourceFd12 2>&1 | java EchoNFd12 5 > temp2.txt
> java SourceFd12 2>&1 | java EchoNFd12 5 > temp3.txt 2>&1
  
```

Exercise: The following command-line does not work. What do you think the user was trying to accomplish? Do you think that what they are trying to accomplish is doable?

```
> java SourceFd12 2> temp.txt | java EchoN 5 > temp.txt 2>&1
```

Exercise: Suppose someone wants to take the contents of a file, modify them, and then write the modified results back into the file. Something like this.

```
> java ToUpperCase < data.txt > data.txt
```

This seems reasonable. Explain why this command-line cannot work. Explain the error message that you get when you try this. How would you accomplish this task? Note: Be careful in your explanation of why the command-line cannot work. It is possible to simultaneously read from and write to a file. Why then can't this command-line work? Also, the following doesn't make things any better.

```
> java ToUpperCase < data.txt >> data.txt
```

Exercise: We mentioned earlier that pipelines are useful for solving problems. Each of the following pipelines does a (somewhat) useful calculation. Briefly describe what each pipeline does. (Don't read off the pipeline from left to right. Figure out a brief, four or five word, description of what it computes.)

```
> java LineNumbers < Readme.txt | java Find pipe
> java OneWordPerLine < Readme.txt | java Find pipe | java CountLines
> java OneWordPerLine < Readme.txt | java LineNumbers | java Find pipe
> java LineNumbers < Readme.txt | java LongestLine | java OneWordPerLine
> java LineNumbers < Readme.txt | java FirstN 16 | java LastLine
> java FirstN 16 < Readme.txt | java LastLine | java OneWordPerLine
> java FirstN 16 < Readme.txt | java LastLine | java Reverse | java OneWordPerLine
> java FirstN 16 < Readme.txt | java LastLine | java OneWordPerLine
```

Exercise: Write a Java filter program called "LineN.java" that prints out the N'th line that it finds in its standard input stream, where N is an optional command-line argument. The default value for N should be 1. After reading N lines the program should stop reading its input stream, close its output stream, and terminate. The program should internally store just one single line of text, not N lines.

Exercise: Write a Java filter program called "TailN.java" that writes to standard output the last `N` lines of text it reads from standard input. The value of `N` should be an optional command-line argument. The default value for `N` should be 10. **Do not** write "TailN.java" to read all of its standard input into a `List` of strings (or an array). That is a very bad strategy. Your program does not know how much data it will receive, so it does not know how much memory is needed to store all the input data. It may be more than what is available to your program on its current computer. Your program should implement a circular list (or array) of size `N`. As lines of text are read from stdin, they should be inserted into the circular list, which retains only the last `N` items inserted into it.

- <https://man7.org/linux/man-pages/man1/tail.1.html>
- https://en.wikipedia.org/wiki/Streaming_algorithm

Modeling failure in a pipeline

The programs "Sink.java", "Null.java", and "Bottom.java" are not filter programs. These three programs do not produce any output. They can act as the terminus of a pipeline. You can also put them in the middle of a pipeline and get unusual results.

The "Sink.java" program reads all of its input, but discards it all, and does not write any output. It acts like the Linux `/dev/null` device.

The "Null.java" program immediately exits with an error code. It does not read any of its input or write any output.

The "Bottom.java" program immediately enters an infinite loop. It does not read any of its input or write any output. It's called "Bottom" because that is the name given to it in Type Theory (it is at the "bottom" of a hierarchy of types; in Java, `Object` is at the top of the reference type hierarchy and `null` is at the bottom).

- https://en.wikipedia.org/wiki/Bottom_type
- https://en.wikipedia.org/wiki/Any_type

Of these three programs, only "Sink.java" has any practical use (that's why Linux has the `/dev/null` device). We will use the other two programs as tools for demonstrating some operating

system concepts and for generating some interesting failure modes.

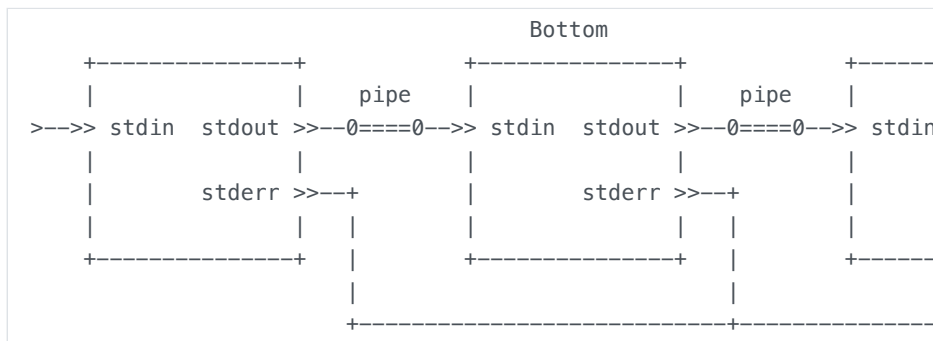
If you want to make your computer work for a while (and waste a bit of energy) run this command-line.

```
> java Source | java EchoN 50000000 | java Sink
```

In theory, that command-line should execute in less than one second, but it probably took much longer. Eventually we will be able to explain why. A command like this is useful for analyzing and understanding the performance of a computer system.

We just saw that `Sink.java` can absorb a lot of data and make it disappear.

If you put `Bottom.java` anywhere in a pipeline, it will make the whole pipeline run forever.

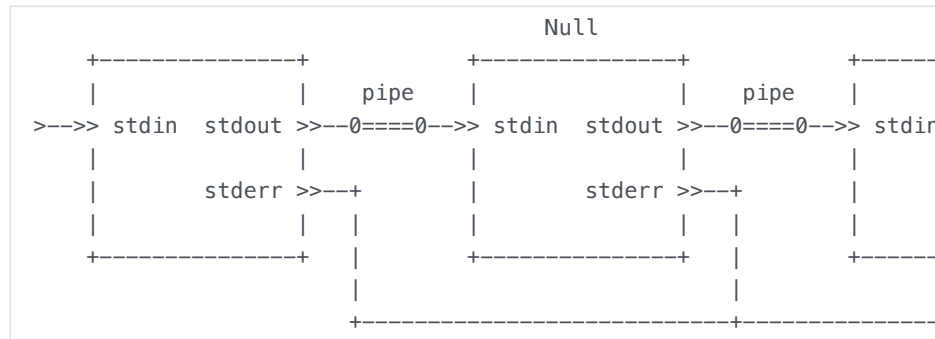


Since `Bottom` does not write any output, but it also does not close its output stream, the process in the pipeline right after `Bottom` will wait forever (block, get stuck) on reading its input data. So that makes the next process act like `Bottom`. But then that process will make the process after it act like `Bottom`, etc.

Now consider the process just before `Bottom` in the pipeline. It may or may not end up acting like `Bottom`. Since `Bottom` does not read any of its input, but it also does not close its input stream, the data written into `Bottom`'s input pipe just accumulates in the pipe's buffer. But the pipe buffer has a fixed size (it is a "bounded buffer"). If the process writing into the pipe (the "producer") fills up the pipe buffer, then that process gets blocked (stuck) when the pipe is full. Since `Bottom` (the "consumer") will never remove anything from the pipe buffer, the buffer will stay full forever, and the producer process will stay blocked forever. At that point, the

producer is acting like `Bottom`. If the process on its left also fills up its pipe buffer, then that process also ends up acting like `Bottom`. (If a process writes less output than the size of the pipe buffer, then that process can terminate normally and does not end up acting like `Bottom`.)

Let us analyze what happens if we put `Null.java` in a pipeline.



The `Null` process immediately throws an error. That causes the process to be terminated. Its input and output streams are then closed by the operating system. The two pipes on either end no longer work (they are de-allocated by the operating system). The process writing to `Null`'s input pipe will throw an error as soon as it tries to write to its output stream (so that process now acts like `Null`). The process reading from `Null`'s output pipe does not get an error when it reads from its standard input stream. Instead, that process see the end-of-file (eof) condition on its input stream. That process can proceed normally without ever knowing that its data source died. So process to the left of `Null` end up acting like `Null` (they crash), and processes to the right of `Null` proceed normally.

Here is an interesting experiment that we can do with

`Bottom.java`. We can use it to show us how big the pipeline buffers are.

```
> java Source | java EchoNFd12 500 | java Bottom
```

When you run this command-line, the text you see in the console window represents the size of a pipeline buffer. The `EchoNFd12` process writes one line to its standard error for every line it writes to its standard output. When its standard output gets blocked because

of the full pipe buffer, the process gets stuck. The lines of text in the console window are equal to the lines of text that are stuck in the pipe buffer.

This command-line lets us see how long it takes for the process in front of `Null` to detect that its output pipe has disappeared.

```
> java Source | java EchoNFd12 50 | java Null
```

Run this command-line several times. You will probably get differing numbers of output lines in the console window. The more lines in the console window, the longer it took `EchoNFd12` to detect the broken pipe on its standard output stream. Sometimes `EchoNFd12` manages to squeeze out all of its output before detecting the broken pipe (but that output all disappears when the pipe is de-allocated).

Modeling network connections with pipelines

A pipe can be thought of as a simple model for a network connection. We can use the `Sink`, `Bottom` and `Null` process to model different kinds of failures in a chain of network connections.

This command models a network connection that stops processing its data, but does not break the network connection. The pipeline just waits until the connection is finally closed.

```
> java CountLines < big.txt | java Sink | java CountWords
```

This command models a network connection that fails. Notice that the end of the pipeline does not notice the failure, it just thinks it was not sent any data. But the beginning of the pipeline detects the failure of its connection.

```
> java CountLines < big.txt | java Null | java CountWords
```

This command models a network connection that stops responding, but does not break the network connection. The pipeline stalls until you kill it (use Ctrl-c).

```
> java CountLines < big.txt | java Bottom | java CountWords
```

In the "filter_programs" folder there is a shell script called "big-text-file.cmd". If you double-click on it, it will download a kind of big (6 MB) text file called "big.txt". For our experiments, "big.txt" is sometimes not big enough. The shell script "bigger-text-file.cmd" doubles the size of "big.txt". By repeatedly double-clicking on "bigger-text-file.cmd" you can grow a *really* big text file.

Using the filter programs in other directories

We have seen that this collection of filter programs is useful for doing experiments and creating demonstrations. To use these programs in other folders, you could copy every filter program that you want to use. But there is a better way to share these programs. In the "filter_programs" folder there are two cmd scripts called "build_classes.cmd" and "build_filters_jar_file.cmd". Double-click on the first to compile all the Java programs to class files. Then double-click on the second to build a jar file containing all the filter programs. Then double-click on "clean_classes.cmd" to delete all the class files, leaving the new jar file. The single file "filters.jar" can be copied to another folder to provide that folder with all the filter programs. (You should open each of these three shell scripts in a text editor to see the code that is written in them.)

To run a filter program from the jar file, we use a slightly different command-line.

```
> java -cp fliters.jar ToUpperCase
```

We need to put the jar file in java's classpath so that the Java Virtual Machine (JVM) can find the class files in the jar file.

If we want to run a pipeline, then each `java` program in the command-line needs to have its classpath set.

```
> java -cp filters.jar Source | java -cp filters.jar EchoN 101 | java -cp filters.jar ToUpperCase
```

This is, of course, tedious and error prone. But there is a solution. The JVM has an environment variable for its classpath. If we use our shell to set the CLASSPATH environment variable, then whenever we run the `java` program in that shell session, the `java` program will have the needed classpath.

```
> set CLASSPATH=.;filters.jar  
> java Source | java EchoN 101 | java CountLines
```

We only need to set CLASSPATH once per shell session.

In the "filter_programs" folder there is another shell script, "filters-with-classpath.cmd". Double-click on that file. It creates a new command-prompt window in whatever folder contains that script file, the new cmd session has CLASSPATH set, and the prompt for the shell is set to a short, helpful string "filters> ". All of that is done with one cmd command-line.

```
cmd /k "set CLASSPATH=.;filters.jar & set PROMPT=filters$G "
```

To use this collection of filter programs in some other folder, copy the two files "filters.jar" and "filters-with-classpath.cmd" into the folder and then double-click on the cmd file. The command-prompt window that pops up is ready to go.

Command-line Syntax

One important idea to remember is that Linux bash, Windows cmd, and Windows PowerShell are really programming languages. They have all the elements of a programming language, variables, conditional expressions, for-loops, functions, and data structures. Windows PowerShell is even an object-oriented language. These are programming languages whose main use is writing programs (usually called "scripts" or "batch files") that control actions performed by the operating system. These languages emphasize things like creating folders and files, moving around in the file system, running programs, doing administrative tasks (creating accounts, installing or configuring software, monitoring performance, backing up files, etc.).

- [https://en.wikipedia.org/wiki/Shell_\(computing\)](https://en.wikipedia.org/wiki/Shell_(computing))
- https://en.wikipedia.org/wiki/Scripting_language

When we type a command at a command-line prompt, we are writing one line of code in one of the these programming languages (cmd, bash, or PowerShell). Typing one command-line at the cmd or bash prompt is pretty much the same thing as typing one line of code at a JShell prompt or a python prompt. Just as each of Java and

Python have a syntax, the languages cmd, bash, and PowerShell each have a syntax. In this section we will look at some aspects of the cmd and bash syntax. The cmd and bash languages have similar syntax (PowerShell is very different from the other two). Most of what we look at in this section is true for both cmd and bash. We will point out places where they differ.

We have seen that command-lines can be made up of, among other things,

- program names,
- command-line arguments,
- file names,
- I/O redirection operators (`<`, `>`, `>>`, and `2>`),
- the pipe operator (the `|` character).

We will look at the syntax of building complex command-lines that combine all of these elements along with a few new elements.

Before looking at specific elements of the command-line syntax, we need to distinguish a couple of ideas. The phrases "command-line" and "command-line argument" are ambiguous. What is a "command-line" or a "command-line argument" depends on a certain context.

Consider the following "command-line" typed into the cmd or bash prompt. It uses the Java program `Find.java` from the "filter_programs" directory.

```
> java Find pipe < Readme.txt > temp.txt
```

The string that comes after the prompt is a "command-line" from the point of view of cmd or bash, but it is *not* a command-line from the point of view of the operating system (either Windows or Linux).

Suppose we wanted to use `ProcessBuilder` to execute this command-line. The following `ProcessBuilder` constructor call is not equivalent to the above command-line.

```

ProcessBuilder pb = new ProcessBuilder("java",
                                       "Find",
                                       "pipe",
                                       "<",
                                       "Readme.txt",
                                       ">",
                                       "temp.txt");

```

This constructor call assumes that the tokens "<", "Readme.txt", ">", and "temp.txt" are command-line arguments to the "java" program. But they are not. Those four tokens are information for the command-line interpreter (either cmd or bash) that tell the interpreter to set up I/O redirection for the "java" program. The sub string "java Find pipe" is an operating system command-line. It instructs the operating system to execute the "java" program with two command-line arguments, "Find" and "pipe".

Here is the correct `ProcessBuilder` constructor call for the above shell command-line (see the "Readme_ProcessBuilder.md" document for details).

```

ProcessBuilder pb = new ProcessBuilder("java",
                                       "Find",
                                       "pipe")
    .redirectInput(new File("Readme.txt")) // <
    .redirectOutput(new File("temp.txt")); // >

```

Notice how the `<` and `>` shell operators are translated into `ProcessBuilder` method calls. Here is code that will execute this command-line from within JShell (try it).

```

var p = new ProcessBuilder("java", "Find", "pipe").
    redirectInput(new File("Readme.txt")).
    redirectOutput(new File("temp.txt")).
    start()

```

Now consider this command-line typed at a cmd or bash prompt.

```
> java Find pipe < Readme.txt | java Find filter > temp.txt
```

This *one* shell command-line is made up of *two* operating system command-lines. The sub string "java Find pipe" and the sub string "java Find filter" are the command-lines that the operating system uses to create two processes. All the other tokens on the shell command-line are used by the shell to figure out the I/O redirections and the pipe between the two processes.

We have two notions of "command-line", the shell's notion of a command-line and the operating system's notion of a command-line.

Let's go back to our original command-line.

```
> java Find pipe < Readme.txt > temp.txt
```

How many "command-line arguments" are there? The answer is, of course, "It depends!" (on the context).

On the one hand, we can say that there are "no command-line arguments" because this is just an input string that the shell process reads from its standard input stream. The shell process parses this string and then builds a command-line to give to the operating system. The command-line for the operating system asks the OS to create a "java" process with two command-line arguments, "Find" and "pipe". The rest of the input string is used by the shell process to decide to ask the OS to redirect the standard input and output streams for the "java" process. From the point of view of the "java" process we can say that there are "two command-line arguments". But there is still a third point of view. The "java" process implements the Java Virtual Machine (JVM) and the "Find.class" file an executable file from the point of view of the JVM. The JVM (virtually) executes a `Find` process. The `main()` method of the `Find` process is passed "one command-line argument", the string "pipe".

So the answer to the question, "How many command-line arguments are there?" is none, from the point of view of the shell process, two from the point of view of the "java" process, and one from the point of view of the (virtual) `Find` process.

Another way to say this is that the tokens "Find" and "pipe" are definitely command-line arguments ("Find" in one way and "pipe" in two ways), and the "java", "<", "Readme.txt", ">", and "temp.txt" tokens are not command-line arguments, they are tokens used by the shell process.

Question: How many command-line arguments are in the following command?

```
> java -cp filters.jar Find pipeline < Readme.txt
```

The operating system syntax for a command-line is very simple. It consists of just a program name followed by any number of arguments.

The syntax for the shell command-line is more complicated. We are not going to present a full grammar for the shell language. Instead, we will describe a simplified grammar for just part of the shell language.

Let us build up a grammar for command-lines. At its most basic, a command-line runs a program.

```
> programName
```

We can give that program a sequence of command-line arguments.

```
> programName arg1 arg2 arg3 arg4
```

We can also give that program some I/O redirections.

```
> programName arg1 arg2 arg3 arg4 < data.txt > results.txt 2>&1
```

Here is a grammar for a simple `command`. A `command` is a program name, followed by a sequence of command-line arguments, followed by a sequence of redirections.

```
command ::= programName [ arg ]* [redirect]*

redirect ::= redirectOp fileName
           | '2>&1'
           | '1>&2'

redirectOp ::= '<' | '>' | '>>' | '2>' | '2>>'
```

We can use the pipe operator to connect two programs in a sequence.

```
> program1 | program2
```

Each of "program1" and "program2" can be given command-line arguments and some I/O redirections. In other words, each of "program1" or "program2" can be replaced by any simple `command` from the previous grammar.

Here is a grammar that adds the pipe operator to the grammar for a simple `command`. A `pipeline` is a sequence of simple commands separated by a pipe operator.

```

pipeline ::= command [ '|' command ]*

command ::= programName [ arg ]* [redirect]*

redirect ::= redirectOp fileName
           | '2>&1'
           | '1>&2'

redirectOp ::= '<' | '>' | '>>' | '2>' | '2>>'

```

Let us add to our grammar a part of the shell language that we have not yet seen. The shell has conditional (boolean) operators, '&&' and '||', just like other programming languages. Consider this command-line.

```
> program1 && program2
```

The idea is to run "program1" and "program2" but only run "program2" if "program1" succeeds. If "program1" fails, then don't run "program2" and the whole command-line fails.

Consider this command-line.

```
> program1 || program2
```

The idea is to run "program1" and determine if it succeeds or fails. If "program1" succeeds, then don't run "program2" and the whole command-line succeeds. If "program1" fails, then run "program2" to see if it succeeds or fails, which determines the status of the whole command-line.

What we just described is the definition of **short circuited** boolean operators. For these boolean shell operators, the boolean value "true" is when a program succeeds, that is, it terminates without an error. The boolean value "false" is when a program fails, that is, it terminates with an error status.

- https://en.wikipedia.org/wiki/Short-circuit_evaluation

You can test these boolean shell operators using the Java programs from the "filter_programs" directory. Run these example command-lines.

```

filters> java Null && echo done
filters> java Null || echo done
filters> java Sink || echo done
filters> java Sink && echo done

```


In a conditional command like the following, we should be able to replace "program1" or "program2" with any simple command, or even with any pipeline command.

```
> program1 || program2
```

This leads to a new grammar that adds the conditional operators at a lower precedence than the pipeline operator. A `conditional` command is a sequence of `pipeline` commands separated by a boolean operator.

```
conditional ::= pipeline [ ('&&' | '||') pipeline ]*

pipeline ::= command [ '|' command ]*

command ::= programName [ arg ]* [redirect]*

redirect ::= redirectOp fileName
           | '2>&1'
           | '1>&2'

redirectOp ::= '<' | '>' | '>>' | '2>' | '2>>'
```

Let us add another element to our grammar. Sometimes we want to execute a sequence of commands without regard to whether any of them succeed or fail. The following command-line says execute "program1" and then execute "program2" and then execute "program3".

```
> program1 & program2 & program3
```

This is the same idea as the sequencing operator in Java, C, or C++, the semi-colon, ';'. But cmd uses the '&' character for sequencing.

In the last command-line, we should be able to replace any one of the programs with a full blown "conditional command".

This leads to a new grammar that adds the sequencing operators at a lower precedence than the conditional (and pipe) operators. A sequenced command is a sequence of `conditional` commands separated by a '&'.

```

command_line ::= conditional [ '&' conditional ]*

conditional ::= pipeline [ ('&&' | '||') pipeline ]*

pipeline ::= command [ '|' command ]*

command ::= programName [ arg ]* [redirect]*

redirect ::= redirectOp fileName
            | '2>&1'
            | '1>&2'

redirectOp ::= '<' | '>' | '>>' | '2>' | '2>>'

```

Exercise: For each command-line below, specify under what condition "program3" will be executed.

```

> program1 & program2 && program3
> program1 && program2 && program3
> program1 & program2 || program3
> program1 || program2 || program3
> program1 || program2 && program3
> program1 || program2 & program3

```

Exercise: Under what condition do you think a pipeline command succeeds or fails? For each command-line below, under what condition will "program3" be executed?

```

> program1 | program2 && program3
> program1 | program2 || program3

```

We need to add one more element to our grammar. The pipe operator has higher precedence than the conditional operators, which have higher precedence than the sequencing operator. We want a way to "group" commands so that we can change the order of operation of these operators.

For example, we want the following command-line to mean, execute "program1" with its output piped into "program3" and then (after "program1" terminates) execute "program2" with its output piped into "program3".

```

> (program1 & program2) | program3

```

Notice that the previous command-line is not the same as the following one, which says, execute "program1" and then sequence that with the execution of the pipeline "program2 | program3", because '|' has higher precedence than '&'.

```
> program1 & program2 | program3
```

Here is our last grammar. It puts the grouping operator in the definition of `command`, so grouping has higher precedence than the pipe, conditional, or sequence operators.

```
command_line ::= conditional [ '&' conditional ]*

conditional ::= pipeline [ ('&&' | '||') pipeline ]*

pipeline ::= command [ '|' command ]*

command ::= programName [ arg ]* [redirect]*
          | '(' command_line ')' [redirect]*

redirect ::= redirectOp fileName
          | '2>&1'
          | '1>&2'

redirectOp ::= '<' | '>' | '>>' | '2>' | '2>>'
```

Here are some example command-lines that you can use in the "filter_programs" directory. The first command-line uses parentheses to force the '&' operator be applied before the '|' operator. The second and third command-lines show the result when the '|' operator is applied before the '&' operator.

```
filters>(java ToUpperCase & java Double)| java EchoN 3
filters> java ToUpperCase & java Double | java EchoN 3
filters> java ToUpperCase &(java Double | java EchoN 3)
```

Exercise: Come up with your own example, using the filters from the "filter_programs" directory, that will demonstrate what this kind of command grouping will do.

```
> ( program1 & program2 & program3 ) > temp.txt
```

What about a command-line like this?

```
> ( program1 & program2 & program3 ) < data.txt
```

Exercise: Come up with example command-lines, using the filter programs from the "filter_programs" directory, to figure out what command-lines like these do.

```
> program < file1 < file2 < file3
> program > file1 > file2 > file3
> program1 > file1 | program2 < file2
> (program1 | program2) > file1 < file2
> program > file1 2>&1 1>&2 2> file2
```

Notice, using the grammar, that these are syntactically correct command-lines. (Hint: For the last example, make sure you use a program that sends data to both its standard output and standard error streams.)

Exercise: Come up with an example command-line that demonstrates what the '`1>&2`' redirection operator does.

The above grammar is by no means complete. For example, consider the following simple command. The quotation marks are not in our grammar. What they do is group characters into a string. That makes "`c d e`" a single (string) command-line argument.

```
> program a b "c d e" f g
```

But what if a program expects a `""` character in one of its command-line arguments? We can **escape** the `""` character so that it is part of a command-line argument, instead of being an operator for the shell process. In the `cmd` shell, the **escape character** is `^` (in Java, the escape character is `\`). The escape character changes the meaning of the character that immediately follows it. In the following command-line, "`program`" has two command-line arguments and the first argument has a quoted sub string.

```
> program "Hello ^"____^", you are our lucky winner!" Bob
```

- https://en.wikipedia.org/wiki/Escape_character#Windows_Command_Prompt

A balanced pair of parentheses may or may not be considered a grouping operator. In the second to last example below, the parentheses do not group anything. They are part of the strings being echoed.

```
> echo hello & echo bye
> (echo hello) & (echo bye)
> (echo hello & echo bye)
> echo (hello & echo bye)
> echo "hello & echo bye"
```

Here are some references for the CMD shell syntax.

- <https://ss64.com/nt/syntax.html>
- <https://ss64.com/nt/syntax-esc.html>
- <https://ss64.com/nt/syntax-conditional.html>

- <https://ss64.com/nt/syntax-redirection.html>
- <https://ss64.com/nt/>
- <https://learn.microsoft.com/en-us/windows-server/administration/windows-commands/windows-commands>
- <https://learn.microsoft.com/en-us/windows-server/administration/windows-commands/command-line-syntax-key>
- <https://learn.microsoft.com/en-us/windows-server/administration/windows-commands/cmd>
- [https://learn.microsoft.com/en-us/previous-versions/cc723564\(v=technet.10\)](https://learn.microsoft.com/en-us/previous-versions/cc723564(v=technet.10))
- [https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-xp/bb490954\(v=technet.10\)](https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-xp/bb490954(v=technet.10))
- [https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-xp/bb490982\(v=technet.10\)](https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-xp/bb490982(v=technet.10))

Here are some references for the bash shell syntax.

- https://man7.org/linux/man-pages/man1/bash.1.html#SHELL_GRAMMAR
- <https://www.gnu.org/software/bash/manual/bash.html#Shell-Syntax>
- <https://catonmat.net/bash-redirections-cheat-sheet>

Here are some review problems that ask you to use the material discussed in this document.

Problem 1: Explain what each of the following possible command-lines mean. In each problem, you need to associate an appropriate meaning to the symbols `a`, `b` and `c`. Each symbol can represent either a program, a file, or a command-line argument.

For example "a is the name of a program, b and c are the names of files", or "a and b are the names of programs and c is the name of a file", or "a is the name of a program, b and c are arguments to the program". Also give a specific example of a runnable command-line with the given format using Windows command-line programs like `dir`, `more`, `sort`, `find`, `echo`, etc.

```

> a > b < c
> a < b > c
> a | b > c
> a < b | c
> a  b  c
> a  b > c
> a  b | c
> a & b < c
> a < b  c
> a < b & c
> a & b | c
> a &(b | c)
>(a & b) | c
>(a & b)> c
> a & b  c
> a & b & c

```

Problem 2: Draw a picture illustrating the processes, streams, pipes, and files in each of the following command-lines.

(a)

```
> b < a | c > d
```

(b)

```
> a < b | c 2> d | e > f 2> d
```

Problem 3: Draw a picture that illustrates all the processes, pipes, files, and (possibly shared) streams in the following situation. A process p1 opens the file a.txt for input and then it opens the file b.txt for output. Then process p1 creates a pipe. Then p1 creates a child process p2 with p2 inheriting a.txt, the pipe's input, and p1's stderr as p2's stdin, stdout and stderr streams. Then p1 creates another child process p3 with p3 inheriting the pipe's output, b.txt, and p1's stderr as p3's stdin, stdout and stderr streams. Then p1 closes its stream to a.txt and the pipe's output.

Problem 4: For the Windows cmd shell, the `dir` command is a builtin (or, internal) command so the `cmd.exe` process does all the work for the directory listing (there is no `dir.exe` program). On the other hand, the `sort` and `find` commands are not builtin (so there are `sort.exe` and `find.exe` programs).

- <https://ss64.com/nt/syntax-internal.html>

For each of the following cmd command-lines, draw a picture of all the relevant processes that shows the difference between a pipeline with a builtin command and a pipeline with non builtin commands.

(a)

```
> dir | find "oops"
```

(b)

```
> sort /? | find "oops"
```

The bash shell also has "builtin" commands. For example, the `cd` command is builtin to bash, just as it is builtin to cmd.

- https://man7.org/linux/man-pages/man1/bash.1.html#SHELL_BUILTIN_COMMANDS

Problem 5: What problem is there with each of the following two command lines? Hint: Try to draw a picture of all the associated processes, streams, pipes, and files.

```
> a | b < c  
> a > b | c
```

Console, terminal, tty

When you begin to read and learn about the command-line, along with the word "shell" you will often see the words "console", "terminal" and "tty". These three words are often used interchangeably, but in some situations they have distinct meanings. In a modern operating system, like Windows or Linux, they refer to the program that the shell interpreter (bash, cmd, or PowerShell), or any other "command-line program", runs in. In a sense, they are the GUI for the command-line interpreter.

Here are some example definitions for these words.

- <https://www.linfo.org/console.html>
- https://www.linfo.org/terminal_window.html
- <https://www.linfo.org/teletype.html>
- https://en.wikipedia.org/wiki/Computer_terminal
- https://en.wikipedia.org/wiki/Terminal_emulator
- <https://learn.microsoft.com/en-us/windows/console/definitions>

Microsoft has good documentation about its modern, open source Windows Terminal program and the Console interface.

- <https://devblogs.microsoft.com/commandline/windows-command-line-background/>
- <https://github.com/Microsoft/Terminal#readme>
- <https://devblogs.microsoft.com/commandline>
- <https://learn.microsoft.com/en-us/windows/console/>
- <https://learn.microsoft.com/en-us/windows/win32/inputdev/about-keyboard-input>

Linux tends to use the term "tty" for a terminal.

- <https://man7.org/linux/man-pages/man1/tty.1.html>
- <https://man7.org/linux/man-pages/man3/termios.3.html>
- <https://man7.org/linux/man-pages/man4/tty.4.html>

Here are a few historical explanations of these terms.

- <https://www.hanselman.com/blog/whats-the-difference-between-a-console-a-terminal-and-a-shell>
- <https://unixdigest.com/articles/the-terminal-the-console-and-the-shell-what-are-they.html>
- <https://bestascitable.com/>
- <https://www.linusakesson.net/programming/tty/>

Terminals, consoles, command-lines, and shells are at the lowest level of a hierarchy of increasingly sophisticated User Interfaces to computers.

- [CLI - Command-line interface](#)
- [TUI - Text-based user interface](#)
- [GUI - Graphical user interface](#)
- [UI - User interface](#)

TUI interfaces are interesting. You can describe them as "text based" but not "line based". Another way to describe them is a GUI where the fundamental pixel is a character instead a dot. So the "framebuffer" (the screen) has the structure of a two-dimensional array of `char`,

```
char[][] framebuffer = char[64][80] // 64 lines with 80 columns
```


instead of the GUI framebuffer which is a two-dimensional array of `Color`.

```
Color[][] framebuffer = Color[1080][1920] // HD resolution
```

Documentation built with [MkDocs](#).

